BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 2

Roll no: 44 PRN No: 12411367 Name of Student: **Sarthak Kokadwar**

Subject Name : Deep Learning

Problem Statement :

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix

Algorithm :

Multilayer Perceptron (MLP) Classification on Iris Dataset

- Import the required Python libraries for dataset handling, preprocessing, model training, evaluation, and visualization.
- Load the Iris dataset from the Scikit-learn library.
- Separate the dataset into input features (X) and target labels (y).
- Split the dataset into training (80%) and testing (20%) sets using train_test_split().
- Standardize the training and testing data using StandardScaler to normalize the feature values.
- Create a Multilayer Perceptron (MLP) classifier with two hidden layers, each containing 10 neurons, using the ReLU activation function and Adam optimizer.
- Train the MLP model using the training dataset with the fit() method.
- Predict the class labels for the test dataset using the trained model.
- Evaluate the model by calculating:

  - Accuracy
  - Precision
  - Recall
  - F1-Score

 Generate the Confusion Matrix to compare actual and predicted classes.
 Display the Confusion Matrix graphically using ConfusionMatrixDisplay.
 Plot the MLP Loss Curve to visualize the reduction in training loss over iterations.
 Perform 10-Fold Cross Validation to evaluate the model's generalization performance.
 Display the Classification Report containing precision, recall, F1-score, and support for each Iris species.

Code :

1. Import Required Libraries :

```
    import numpy as np
import matplotlib.pyplot as plt

from sklearn import datasets
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    ConfusionMatrixDisplay
)
```

2. Load the Iris Dataset :

```
iris = datasets.load_iris()
X = iris.data
y = iris.target
```

3. Split the Dataset :

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

4. Feature Scaling:

```
   scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

5. Create the MLP Classifier :

```
mlp = MLPClassifier(
    hidden_layer_sizes=(10, 10),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)
```

6. Train the Model :

```
mlp.fit(X_train, y_train)
```

7. Predict the Test Data:

```
y_pred = mlp.predict(X_test)
```

8. Calculate Performance Metrics:

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
```

9. Generate the Confusion Matrix :

```
cm = confusion_matrix(y_test, y_pred)
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=iris.target_names
)

disp.plot(
    cmap='viridis',
    values_format='d',
    ax=axes[0]
)

axes[0].set_title("Confusion Matrix")


axes[1].plot(mlp.loss_curve_)
axes[1].set_title("MLP Loss Curve")
axes[1].set_xlabel("Iterations")
axes[1].set_ylabel("Loss")

plt.suptitle(title)
plt.tight_layout()
plt.show()
```

10. K-Fold cross validation :

```
cv_scores = cross_val_score(
        mlp,
        X_train,
        y_train,
        cv=10,
        scoring='accuracy'
    )
```

11 Display Results:

```
print("========== RESULTS ==========\n")

print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
```

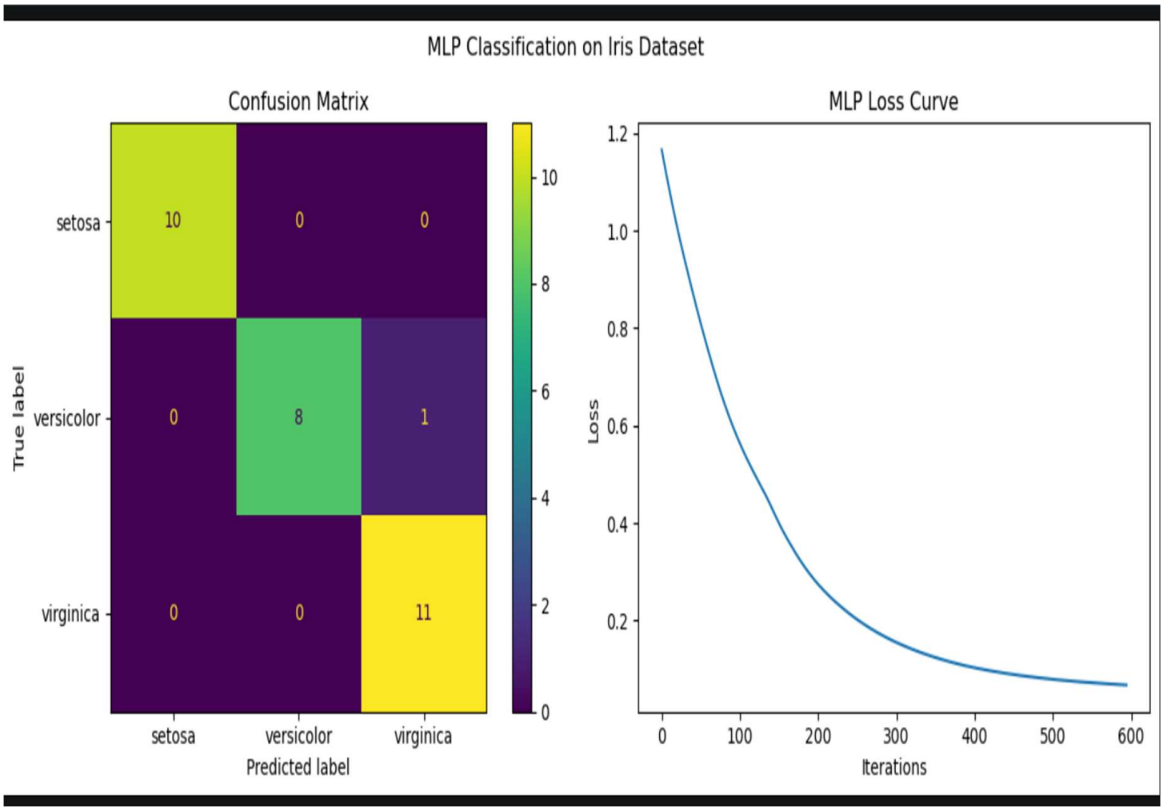
```python
print(f"Recall    : {recall:.4f}")
print(f"F1 Score  : {f1:.4f}")

print("\n10-Fold Cross Validation Accuracy:")
print(cv_scores)
print(f"Average Accuracy: {np.mean(cv_scores):.4f}")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:\n")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```

- **OUTPUT :**



```
========== RESULTS ==========

Accuracy  : 0.9667
Precision : 0.9694
Recall    : 0.9667
F1 Score  : 0.9664

10-Fold Cross Validation Accuracy:
[0.91666667 1.         1.         1.         0.66666667 0.83333333
 1.         1.         1.         0.91666667]
Average Accuracy: 0.9333

Confusion Matrix:
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

Classification Report:

              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.89      0.94         9
   virginica       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```

- **Conclusion:**

The Multilayer Perceptron (MLP) classifier was successfully implemented for the classification of the Iris dataset. The dataset was preprocessed using feature scaling and divided into training and testing sets before training the model. The trained MLP achieved an **accuracy of 96.67%**, demonstrating excellent classification performance. The confusion matrix showed that almost all flower samples were classified correctly, with only one Versicolor sample misclassified as Virginica. The loss curve indicated a steady decrease in training loss, confirming that the model learned effectively and converged successfully. Additionally, the 10-fold cross-validation results demonstrated

that the model generalizes well to unseen data. Overall, the MLP classifier proved to be an accurate and reliable model for classifying Iris flower species.